

旭丰新材料 — AI 智能客服系统

基于 [AgentScope](#) 框架构建的全栈 AI 智能客服系统，为黄石旭丰新材料科技有限公司（模具钢、高速钢等特钢产品）提供官网智能客服、产品导航、深度需求分析和多智能体协同服务。

目录

- 1. [项目概览](#)
- 2. [系统架构](#)
- 3. [AgentScope 框架使用](#)
- 4. [后端详解](#)
- 5. [前端详解](#)
- 6. [Agent 角色体系](#)
- 7. [技能 \(Skills\) 体系](#)
- 8. [工具 \(Tools\) 体系](#)
- 9. [API 接口文档](#)
- 10. [测试](#)
- 11. [部署](#)
- 12. [开发指南](#)

1. 项目概览

1.1 业务背景

黄石旭丰新材料科技有限公司（前身：汇隆特钢）位于湖北黄石山南工业园，年产 4 万余吨模具钢、高速工具钢。拥有 ISO 9001、ISO 14001 等认证。公司需要一套 AI 智能客服系统来处理网站访客的产品咨询、价格询价、技术问题等。

1.2 技术栈

层级	技术	说明
AI 框架	AgentScope >= 1.0.14	Agent 编排、工具调用、MCP 集成
模型	DeepSeek Chat	通过 OpenAI 兼容接口调用
后端	FastAPI + Uvicorn	REST API + SSE 流式
前端	React 19 + TypeScript + Vite	SPA 聊天应用
MCP	Streamable HTTP	外部工具集成 (web-search)
容器化	Docker + docker-compose	生产部署

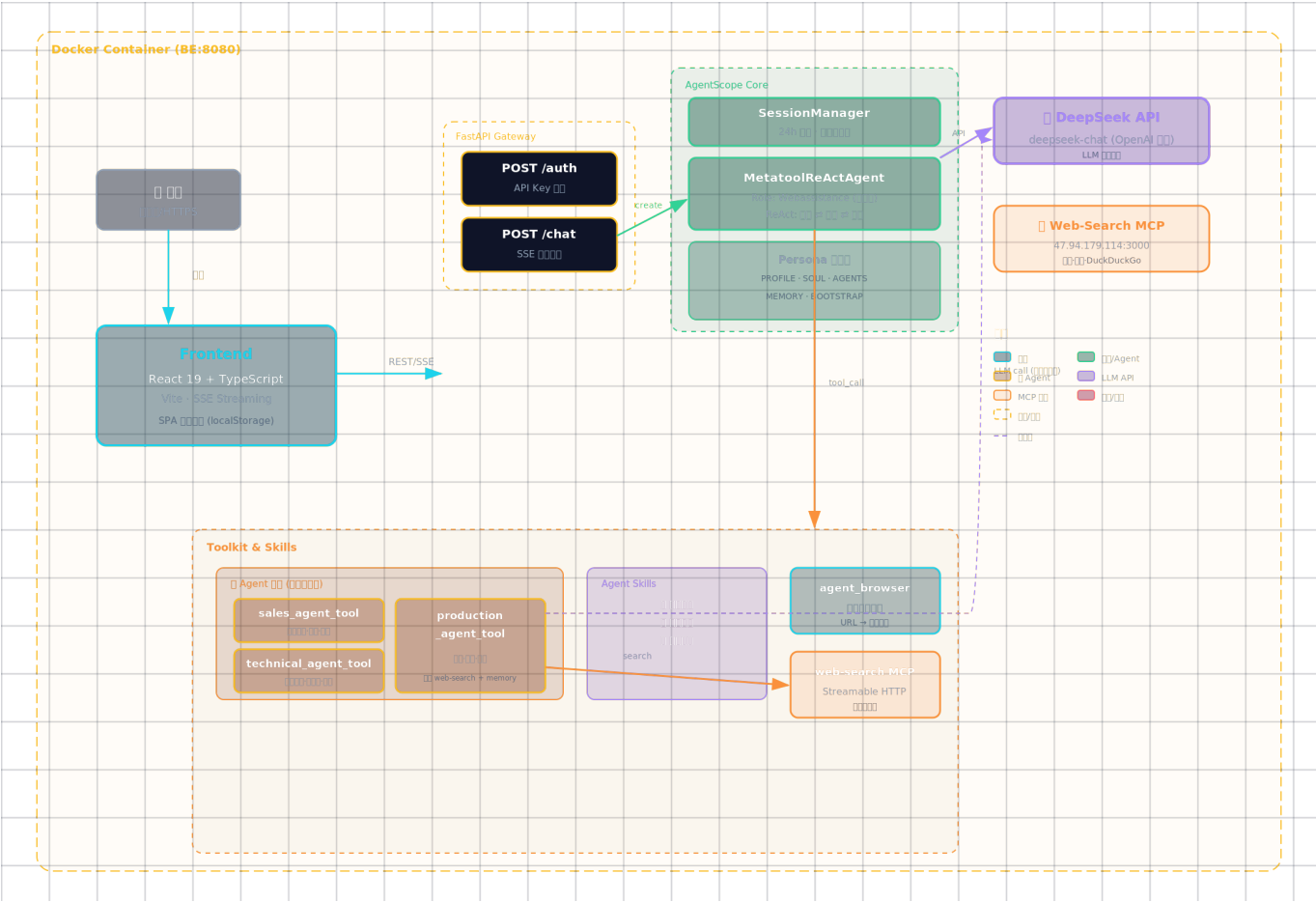
1.3 目录结构

```
testproject/
├── PROJECT.md                    # 本文档
├── .gitignore
├── BE/                          # 后端 (Python)
│   ├── requirements.txt
│   ├── Dockerfile
│   ├── docker-compose.yml
│   ├── API.md                  # API 接口文档
│   ├── .env                    # 环境变量 (gitignore)
│   ├── agents/                # Agent 角色定义
│   │   ├── Webassistance/     # 总AI调度 (官网客服)
│   │   │   ├── PROFILE.md     # 身份与用户画像
│   │   │   ├── SOUL.md        # 核心行为原则
│   │   │   ├── AGENTS.md      # 工作规则与产品数据
│   │   │   ├── MEMORY.md      # 长期记忆
│   │   │   └── BOOTSTRAP.md   # 首次引导
│   │   ├── sales/             # 销售智能体
│   │   ├── technical-engineer/ # 技术工程师智能体
│   │   ├── production/        # 生产流程智能体
│   │   ├── coder/             # 编程助手 (备选角色)
│   │   └── default/           # 通用助手 (备选角色)
│   ├── skills/                # Agent 技能
│   │   ├── company-intelligent-analysis/SKILL.md # 公司智能分析
│   │   ├── memory/SKILL.md    # 长期记忆管理
│   │   └── xufeng-material-navigator/SKILL.md    # 产品官网导航
│   ├── src/                   # 源代码
│   │   ├── app.py             # FastAPI 应用入口
│   │   ├── agent.py           # Agent 会话管理
│   │   ├── config.py          # 配置管理
│   │   ├── prompt_loader.py   # 提示词加载器
│   │   ├── tools/             # 工具实现
│   │   │   ├── agent_browser.py # 官网页面导航工具
│   │   │   └── sub_agent_tools.py # 三个专业子智能体工具
│   │   └── Mcp/               # MCP 客户端
│   │       └── web_search.py  # Web-search MCP 客户端
│   ├── test/                  # 测试
│   │   └── test_agent.py      # API 集成测试
├── FE/                          # 前端 (React)
│   ├── README.md
│   └── my-chat-app/
│       ├── index.html
│       ├── package.json
│       ├── vite.config.ts
│       └── tsconfig.json
```

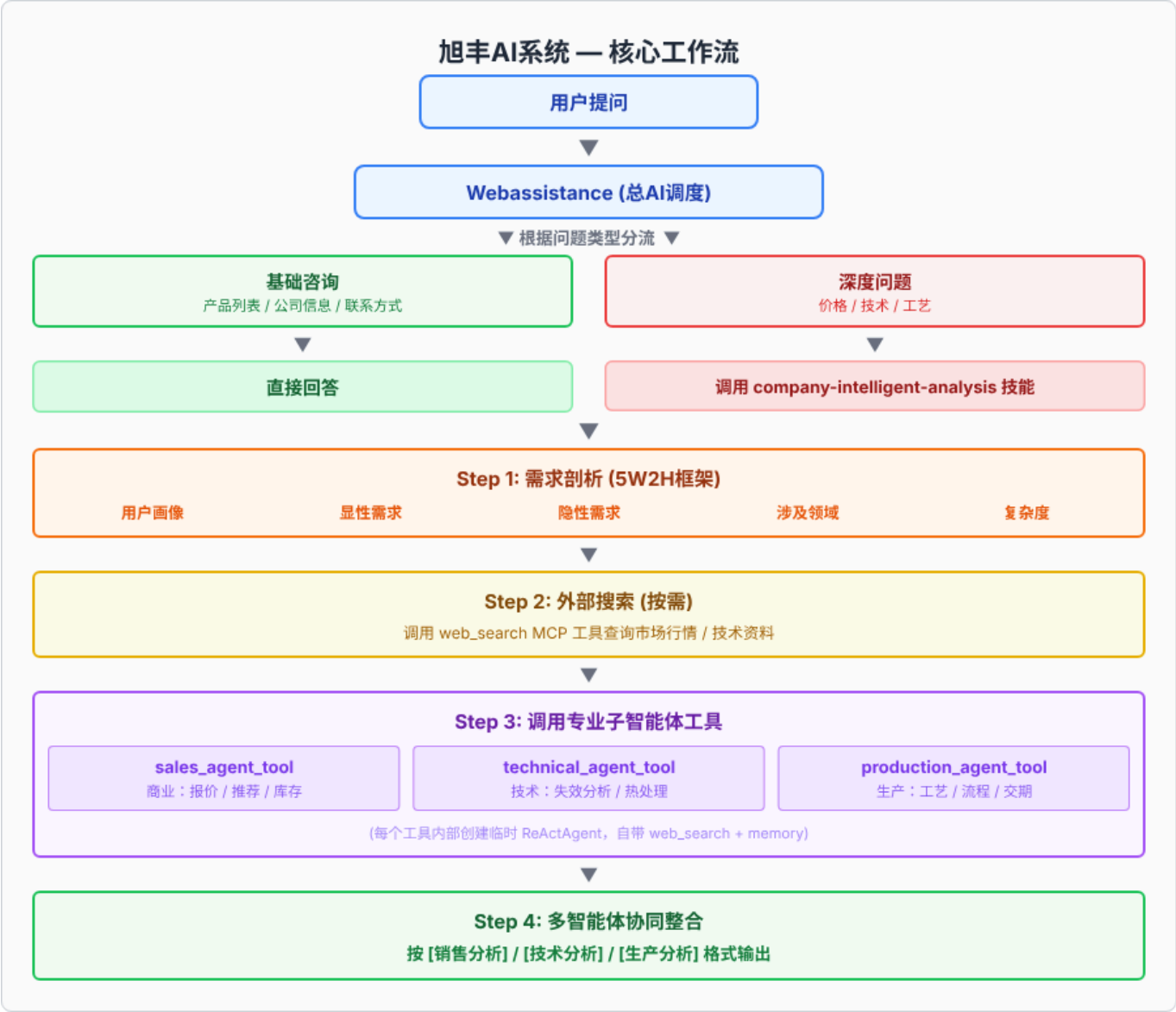
src/	
main.tsx	# React 入口
App.tsx	# 应用主体
App.css	# 样式
api.ts	# API 封装
index.css	# 基础样式
src/	# 废弃（空目录）

2. 系统架构

2.1 整体架构图



2.2 核心 workflow



3. AgentScope 框架使用

3.1 AgentScope 简介

AgentScope 是阿里巴巴 ModelScope 团队开源的 AI Agent 框架，提供：

- **ReActAgent** — 基于 ReAct (Reasoning + Acting) 模式的 Agent，支持工具调用循环
- **Toolkit** — 工具注册与管理容器
- **MCP 集成** — 原生支持 MCP (Model Context Protocol) 协议
- **多种模型** — 支持 OpenAI、DashScope、Ollama 等模型后端
- **技能系统** — 基于 markdown 的 Agent Skill 定义
- **流式输出** — 支持逐 token 流式返回

3.2 核心类与 API

3.2.1 ReActAgent

最核心的 Agent 类，实现 ReAct 推理-行动循环。

```
from agentscope.agent import ReActAgent
from agentscope.model import OpenAIChatModel
from agentscope.memory import InMemoryMemory
from agentscope.tool import Toolkit
from agentscope.formatter import OpenAIChatFormatter
from agentscope.message import Msg

# 创建 Agent
agent = ReActAgent(
    name="MyAgent",                # Agent 名称
    model=OpenAIChatModel(        # LLM 模型
        model_name="deepseek-chat",
        api_key="sk-xxx",
        client_kwargs={
            "base_url": "https://api.deepseek.com",
            "timeout": 60,
            "max_retries": 3,
        },
        stream=True,               # 是否流式输出
    ),
    sys_prompt="你是一个有用的助手", # 系统提示词
    memory=InMemoryMemory(),        # 短期记忆（对话历史）
    toolkit=toolkit,                # 工具包
    formatter=OpenAIChatFormatter(), # 消息格式化器
)

# 发送消息并获取回复
msg = Msg("user", "你好", "user")
reply = await agent(msg)
print(reply.get_text_content())
```

构造函数参数：

参数	类型	必填	说明
name	str	是	Agent 名称
model	OpenAIChatModel	是	LLM 模型实例
sys_prompt	str	是	系统提示词
memory	MemoryBase	是	短期记忆（对话历史存储）
toolkit	Toolkit	是	工具注册容器
formatter	FormatterBase	是	消息格式化器（将 AgentScope Msg 转为模型输入格式）

执行流程（ReAct 循环）：

1. 用户发送消息
 2. Agent 将消息加入 memory
 3. formatter 将 memory 中的消息格式化为模型输入

4. 模型返回响应（可能是文本，也可能是 `tool_call`）
5. 如果是 `tool_call` → 调用 `toolkit` 执行工具 → 工具结果加入 `memory` → 回到步骤 3
6. 如果是文本 → 返回最终回复

3.2.2 OpenAIChatModel

OpenAI 兼容的模型客户端。

```
from agentscope.model import OpenAIChatModel

model = OpenAIChatModel(
    model_name="deepseek-chat",          # 模型名
    api_key="sk-xxx",                    # API Key
    client_kwargs={                       # 传给 OpenAI 客户端的参数
        "base_url": "https://api.deepseek.com",
        "timeout": 60,
        "max_retries": 3,
    },
    stream=True,                          # 是否流式输出
)
```

关键参数：

- `model_name` — 模型标识符（如 `deepseek-chat`、`gpt-4o`）
- `api_key` — API 密钥
- `client_kwargs` — 传递给 `openai.AsyncOpenAI()` 的额外参数
 - `base_url` — API 基础 URL（DeepSeek 兼容 OpenAI 接口）
 - `timeout` — 请求超时时间（秒）
 - `max_retries` — 最大重试次数
- `stream` — `True` 启用流式输出（逐 token 返回），`False` 等待完整响应
- `generation_kwargs` — 生成参数（`temperature`、`max_tokens` 等）

3.2.3 Toolkit

工具注册与管理容器。

```
from agentscope.tool import Toolkit

toolkit = Toolkit()

# 方式1: 注册 Python 函数作为工具
toolkit.register_tool_function(my_async_function)

# 方式2: 注册 MCP 客户端（批量注册 MCP 服务器的所有工具）
from agentscope.mcp import HttpStatelessClient

mcp_client = HttpStatelessClient(
    name="my_mcp",
```

```
        transport="streamable_http",
        url="http://example.com/mcp",
    )
    await toolkit.register_mcp_client(mcp_client)

# 方式3: 注册 Agent Skill (基于目录的 markdown 技能定义)
toolkit.register_agent_skill("skills/my-skill/")

# 获取所有工具的 JSON Schema (用于 LLM function calling)
schemas = toolkit.get_json_schemas()

# 调用工具
from agentscope.message import ToolUseBlock

tool_call = ToolUseBlock(
    type="tool_use",
    id="call_001",
    name="my_function",
    input={"param1": "value1"},
)
result = await toolkit.call_tool_function(tool_call)

# 移除工具
toolkit.remove_tool_function("function_name")
await toolkit.remove_mcp_clients(client_names=["my_mcp"])

# 获取 Agent Skill 的提示词
skill_prompt = toolkit.get_agent_skill_prompt()
```

Toolkit 方法一览：

方法	说明
register_tool_function(fn)	注册 Python 异步函数为工具
register_mcp_client(client)	注册 MCP 客户端（批量导入其所有工具）
register_agent_skill(dir_path)	注册 Agent Skill 目录
get_json_schemas()	获取所有工具的 OpenAI function calling JSON Schema
get_agent_skill_prompt()	获取所有注册 Skill 的提示词拼接文本
call_tool_function(tool_use_block)	执行指定工具
remove_tool_function(name)	移除单个工具
remove_mcp_clients(client_names)	移除 MCP 客户端及其关联工具

3.2.4 ToolResponse

工具函数的返回类型。

```
from agentscope.tool import ToolResponse
from agentscope.message import TextBlock

async def my_tool(param: str) -> ToolResponse:
    """工具描述 - 这会成为 LLM function calling 的 description.

    Args:
        param: 参数描述 - 会成为 function schema 的参数定义

    Returns:
        ToolResponse: 工具执行结果
    """
    return ToolResponse(
        content=[
            TextBlock(type="text", text=f"结果: {param}"),
        ],
        metadata={"key": "value"}, # 可选, 传递给前端的元数据
        is_last=True,             # 是否为最后一块 (流式场景)
    )
```

3.2.5 MCP 客户端 (HttpStatelessClient / HttpStatefulClient)

AgentScope 原生支持 MCP (Model Context Protocol), 允许 Agent 调用外部 MCP 服务器的工具。

```
from agentscope.mcp import HttpStatelessClient, HttpStatefulClient

# 无状态客户端 (每次调用创建临时会话, 用完即断)
stateless = HttpStatelessClient(
    name="web_search",
    transport="streamable_http", # 或 "sse"
    url="http://47.94.179.114:3000/mcp",
)

# 有状态客户端 (保持持久会话, 需要手动 connect/close)
stateful = HttpStatefulClient(
    name="mcp_stateful",
    transport="streamable_http",
    url="https://mcp.example.com/mcp",
)
await stateful.connect() # 建立持久连接
# ... 使用工具 ...
await stateful.close()  # 关闭连接
```

两种客户端对比:

特性	HttpStatelessClient	HttpStatefulClient
连接模式	按需连接, 用完即断	持久连接

特性	HttpStatelessClient	HttpStatefulClient
资源占用	低	高
适用场景	偶尔调用	频繁调用
connect/close	不需要	必须手动管理
StdIO 支持	不支持	支持

按函数级别使用 MCP 工具：

```
# 只获取某一个工具的可调用对象（不注册整个客户端）
func_obj = await stateless_client.get_callable_function(
    func_name="search",
    wrap_tool_result=True,    # True → 返回 ToolResponse, False → 返回原始结果
)

# 直接调用工具
result = await func_obj(query="H13 模具钢 价格")
```

重要： 多个有状态客户端关闭时应按 LIFO（后进先出）顺序，避免错误。

3.2.6 Msg 消息类型

```
from agentscope.message import Msg, TextBlock, ToolUseBlock,
ToolResultBlock

# 用户消息
user_msg = Msg("user", "你好", "user")

# 系统消息
sys_msg = Msg("system", "系统提示", "system")

# 获取文本内容
text = reply.get_text_content()

# 获取消息 metadata
metadata = reply.metadata # dict 或 None

# ToolUseBlock - 工具调用请求
tool_call = {
    "type": "tool_use",
    "id": "call_abc123",
    "name": "web_search",
    "input": {"query": "H13价格"},
}

# ToolResultBlock - 工具执行结果
tool_result = {
    "type": "tool_result",
```

```

        "id": "call_abc123",
        "name": "web_search",
        "output": [...],
    }

# TextBlock – 文本块
text_block = TextBlock(type="text", text="回复内容")

```

3.2.7 MetatoolReActAgent（自定义扩展）

本项目在 `agent.py` 中定义了 `MetatoolReActAgent`，继承 `ReActAgent` 并覆盖 `_acting()` 方法：

```

class MetatoolReActAgent(ReActAgent):
    async def _acting(self, tool_call: ToolUseBlock) -> dict | None:
        """执行工具调用，并将 ToolResponse.metadata 保存到 Msg.metadata 上，
        以便流式输出时能传递给前端（如 page_navigation action）。"""
        # ... 调用父类逻辑 + metadata 转发

```

扩展目的： 将工具执行过程中产生的 `ToolResponse.metadata` 传递给前端。例如 `agent_browser` 工具的 `page_navigation` action 会通过 SSE `action` 事件发送给前端，前端据此导航用户浏览器到对应产品页面。

3.2.8 InMemoryMemory

短期对话记忆，存储对话历史。

```

from agentscope.memory import InMemoryMemory

memory = InMemoryMemory()
# agent 会自动将每条消息添加到 memory
# 对话历史随 session 销毁而丢失

```

3.2.9 OpenAIChatFormatter

将 AgentScope 的消息格式转换为 OpenAI Chat Completions API 格式。

```

from agentscope.formatter import OpenAIChatFormatter

formatter = OpenAIChatFormatter()
# 自动将 Msg 列表转换为 [{"role": "system", "content": "..."}, ...]

```

3.3 提示词加载机制

本项目通过 `prompt_loader.py` 实现基于文件的提示词管理：

```

from prompt_loader import (
    load_agent_prompt,      # 加载角色提示词
    get_memory_path,        # 获取 MEMORY.md 路径
    get_memory_context_prompt, # 生成 memory 路径上下文
    list_available_roles,    # 列出可用角色
    AGENTS_DIR,             # agents/ 目录路径
    SKILLS_DIR,             # skills/ 目录路径
)

# 加载 sales 角色的提示词
system_prompt = load_agent_prompt("sales")
# 返回: PROFILE.md + "\n\n---\n\n" + SOUL.md + "\n\n---\n\n" + AGENTS.md

```

加载规则:

- 加载文件顺序: **PROFILE.md** → **SOUL.md** → **AGENTS.md**
- 分隔符: **\n\n---\n\n**
- 不加载: **MEMORY.md**、**BOOTSTRAP.md**
- 空文件自动跳过

3.4 技能 (Skill) 系统

AgentScope 的 Skill 系统基于目录结构:

```

skills/
  my-skill/
    SKILL.md    # 技能定义 (含 frontmatter 元数据)

```

SKILL.md 格式:

```

---
name: my-skill
description: 技能描述 (会注入到 Agent 的系统提示词中)
---

# 技能标题

## 触发条件
- 条件1
- 条件2

## 执行流程
### 第 1 步: ...
### 第 2 步: ...

```

注册方式:

```
toolkit.register_agent_skill("skills/my-skill/")
```

注册后，`toolkit.get_agent_skill_prompt()` 会返回所有已注册技能的提示词拼接文本，该文本会被追加到 Agent 的系统提示词中。

3.5 工具函数编写规范

一个标准的 AgentScope 工具函数：

```
async def tool_name(param1: str, param2: int = 0) -> ToolResponse:
    """工具简短描述。

    详细描述 - 这会成为 LLM function calling 的 description 字段。
    说明工具的用途、适用场景和使用建议。

    Args:
        param1: 参数1的描述
        param2: 参数2的描述（可选参数必须给默认值）

    Returns:
        ToolResponse: 工具执行结果
    """
    # 工具逻辑...
    return ToolResponse(
        content=[TextBlock(type="text", text="结果文本")],
        is_last=True,
    )
```

要点：

- 函数必须是 `async def`
- docstring 会被解析为 function calling schema 的 `description`
- 参数类型注解会被解析为 schema 的 `parameters`
- 返回值必须是 `ToolResponse`

4. 后端详解

4.1 FastAPI 应用 (app.py)

应用入口，配置 CORS 中间件和路由。

核心端点：

方法	路径	说明
GET	/health	健康检查
POST	/auth	API Key 认证，创建会话

方法	路径	说明
POST	/chat	同步对话
POST	/chat/stream	SSE 流式对话
POST	/logout	销毁会话

SSE 事件类型：

```
// 文本事件 - 累积更新 Agent 回复
{ type: "text", msg_id: "abc123", content: "回复文本" }

// Action 事件 - 触发前端行为
{ type: "action", action: "page_navigation", payload: { url:
"/products/hot-work/h13/", content: "H13..." } }
```

4.2 会话管理 (agent.py)

AgentSession — 持有单个 Agent 实例的会话对象：

```
# 异步工厂方法创建（因为 MCP 注册是异步的）
session = await AgentSession.create(session_id, user_id)

# 会话到期检查
if session.is_expired():
    # 重新认证
    pass

# 更新最后访问时间
session.touch()
```

SessionManager — 管理所有活跃会话：

```
manager = SessionManager()

# 创建会话
session = await manager.create_session(api_key, user_id)

# 获取会话
session = manager.get_session(session_id) # 自动检查过期

# 销毁会话
manager.remove_session(session_id)

# 定期清理过期会话
cleaned = manager.cleanup_expired_sessions() # 返回清理数量
```

会话生命周期:

1. 前端 POST `/auth` → 创建 AgentSession → 返回 session_id
2. 同一 session_id 多次调用 `/chat` → Agent 保持对话上下文
3. 空闲超过 24 小时 → 自动过期
4. 前端 POST `/logout` → 主动销毁

4.3 配置管理 (config.py)

```
class Config:
    # DeepSeek 模型配置 (兼容 OpenAI API)
    DEEPSEEK_API_KEY = os.getenv("DEEPSEEK_API_KEY", "")
    DEEPSEEK_BASE_URL = os.getenv("DEEPSEEK_BASE_URL",
    "https://api.deepseek.com")
    MODEL_NAME = os.getenv("MODEL_NAME", "deepseek-chat")

    # 服务端口
    PORT = int(os.getenv("PORT", 8080))

    # Agent 角色 (对应 agents/ 下的目录名)
    AGENT_ROLE = os.getenv("AGENT_ROLE", "default")

    # 前端认证 Key
    VALID_API_KEYS = {
        "sk-frontend-001": {"user_id": "user_001", "expire_days": 30},
        "sk-frontend-002": {"user_id": "user_002", "expire_days": 30},
    }

    # 会话过期时间
    SESSION_EXPIRE_HOURS = 24
```

环境变量 (通过 .env 文件设置):

```
DEEPSEEK_API_KEY=sk-xxxxxxx
DEEPSEEK_BASE_URL=https://api.deepseek.com
MODEL_NAME=deepseek-chat
AGENT_ROLE=Webassistance
PORT=8080
```

4.4 提示词加载器 (prompt_loader.py)

```
from prompt_loader import load_agent_prompt, get_memory_path,
list_available_roles

# 加载 Webassistance 角色的系统提示词
prompt = load_agent_prompt("Webassistance")
# 返回: PROFILE.md 内容 + "\n\n---\n\n" + SOUL.md 内容 + "\n\n---\n\n" +
```

```
AGENTS.md 内容

# 列出所有可用角色
roles = list_available_roles()
# 返回: ["Webassistance", "coder", "default", "production", "sales",
"technical-engineer"]

# 获取角色 MEMORY.md 路径
path = get_memory_path("sales")
# 返回: "/absolute/path/to/agents/sales/MEMORY.md"
```

5. 前端详解

5.1 技术栈

技术	版本	用途
React	19.2	UI 框架
TypeScript	6.0	类型安全
Vite	8.0	构建工具
TailwindCSS	4.2	样式（依赖已装，实际使用手写 CSS）

5.2 核心组件

App.tsx — 主应用组件，管理全局状态：

```
// 状态管理
const [sessions, setSessions] = useState<Session[]>([]); // 会话列表
const [activeId, setActiveId] = useState<string | null>(null); // 当前活跃会话
const [showAuth, setShowAuth] = useState(false); // 认证弹窗
const [sidebarOpen, setSidebarOpen] = useState(true); // 侧边栏开关

// 核心流程
// 1. 新建会话 → showAuth → 选API Key → auth() → 获得 session_id
// 2. 发送消息 → streamChat(sessionId, message) → SSE 解析 → 更新消息气泡
// 3. 切换会话 → setActiveId → 恢复消息历史
// 4. 删除会话 → logout(sessionId) → 从列表移除
```

子组件：

- **Sidebar** — 左侧会话列表（深色面板，可折叠）
- **ChatArea** — 聊天区域 + 输入框 + 快捷提示
- **MessageBubble** — 消息气泡（用户蓝色/右，AI灰色/左，流式光标动画）
- **AuthModal** — API Key 选择弹窗

5.3 API 封装 (api.ts)

```
// 认证
async function auth(apiKey: string): Promise<SessionInfo>

// 流式聊天（异步生成器）
async function* streamChat(
  sessionId: string,
  message: string,
): AsyncGenerator<StreamMessage>

// 登出
async function logout(sessionId: string): Promise<void>
```

流式解析逻辑：

- 1. Fetch POST /chat/stream
- 2. 获取 response.body 的 reader
- 3. 逐行解析 data: ... 格式
- 4. JSON.parse 提取文本或 action
- 5. data: [DONE] 停止

消息类型：

```
type StreamMessage =
  | { type: 'text'; msg_id?: string; content: string }
  | { type: 'action'; action: string; payload: Record<string, unknown> }
```

5.4 数据持久化

所有会话和消息存储在 localStorage，刷新页面不丢失。

6. Agent 角色体系

6.1 角色定义文件

每个角色包含 5 个 markdown 文件：

文件	加载到系统提示词	用途
PROFILE.md	是	身份定义（名字、定位、风格）和用户画像
SOUL.md	是	核心行为原则（准则、边界、风格）
AGENTS.md	是	工作规则、领域知识、工具使用说明
MEMORY.md	否	长期记忆存储（由 memory 技能维护）

文件	加载到系统提示词	用途
BOOTSTRAP.md	否	首次运行引导（完成后应删除）

6.2 Webassistance（总AI调度）

身份： 旭丰小助手 — 官网 AI 客服，B2B 工业品助手

职责：

- 基础信息咨询（产品列表、公司信息、联系方式）
- 产品官网导航（自动调用 agent_browser 打开产品详情页）
- 深度问题转交（调用 company-intelligent-analysis 技能）

工作规则：

- 只回答公司相关问题
- 中英双语支持
- 不编造信息
- 复杂问题不自己回答，转交专业智能体

可用工具： 所有注册工具和技能

6.3 sales（销售智能体）

身份： 旭丰销售顾问 — 10 年特钢行业销售经验

职责：

- 产品推荐（按应用场景推荐最合适的牌号）
- 价格查询（市场参考价 + 成本推算）
- 库存查询（引导联系销售经理确认）
- 报价单生成（含完整格式模板）
- 客户跟进（主动挖掘需求）

关键数据：

- DC53 成本推算公式（原料成本 + 冶炼加工费）
- 32 款产品的市场参考价格区间
- 压铸/冲裁/切削等场景的选型推荐表

自有工具： web_search MCP + memory skill

环境变量切换： AGENT_ROLE=sales

6.4 technical-engineer（技术工程师智能体）

身份： 旭丰技术顾问 — AI 材料工程师

职责：

- 失效分析（H13 淬火开裂、模具磨损、断裂等）
- 热处理工艺建议（淬火/回火/退火参数）

- 材料性能对比（DC53 vs SKD11、8407 vs H13 等）
- 硬度问题分析
- 钢材选型原理（热作模具钢决策树）

关键数据：

- H13 淬火开裂 6 大原因及完整检查清单
- 8 种常用模具钢的热处理工艺参数表
- ESR vs 非ESR 性能对比表

自有工具： web_search MCP + memory skill

环境变量切换： `AGENT_ROLE=technical-engineer`

6.5 production（生产流程智能体）

身份： 旭丰生产顾问 — 20 年特钢工厂管理经验

职责：

- 生产工艺解释（EAF → LF → VD → ESR → 锻造 → 退火 → 机加工）
- 流程检查与参数验证
- 异常参数提醒
- 生产记录生成
- 交期预估

关键数据：

- 7 道工序的详细工艺参数（温度、时间、压力、质量指标）
- 生产周期预估表（常规 12-18 天，ESR 15-23 天，非标 25-35 天）
- 常见工艺问题解答

自有工具： web_search MCP + memory skill

环境变量切换： `AGENT_ROLE=production`

6.6 coder（编程助手）

备选角色，用于编程开发场景。`AGENT_ROLE=coder`

6.7 default（通用助手）

备选角色，默认通用 AI 助手。`AGENT_ROLE=default`（默认值）

6.8 如何切换角色

```
# 方式1：修改 .env 文件
echo "AGENT_ROLE=sales" >> BE/.env

# 方式2：Docker 环境变量
docker run -e AGENT_ROLE=sales ...
```

```
# 方式3: docker-compose.yml 中设置
environment:
  - AGENT_ROLE=${AGENT_ROLE:-sales}
```

7. 技能 (Skills) 体系

7.1 company-intelligent-analysis（公司智能分析）

触发条件：

- 询问价格、报价、性价比
- 询问选型建议
- 询问材料性能对比、热处理、失效分析
- 询问生产工艺、质量检验、交期
- 明确采购意向
- 跨领域复杂问题

执行流程：

Step 1: 需求剖析

- ├— 用户画像：采购？工程师？模具厂老板？
- ├— 显性需求：用户直接说了什么
- ├— 隐性需求：询价→可能想采购，问选型→可能准备下单
- ├— 涉及领域：商业/技术/工艺/跨领域
- └— 复杂度：简单/中等/复杂

Step 2: 外部搜索（按需）

调用 web_search 获取市场行情/技术资料/工艺标准

Step 3: 调用子智能体工具

sales_agent_tool / technical_agent_tool / production_agent_tool

Step 4: 多智能体协同整合

按【销售分析】 / 【技术分析】 / 【生产分析】 格式输出

调用示例：

用户: "H13什么价格？"

主Agent: 调用 company-intelligent-analysis 技能

- 剖析：【采购意向】【询价】【可能需要对比性能】
- 搜索: web_search("H13 模具钢 价格 2026")
- 调用: sales_agent_tool(question="用户询H13价格...", context="搜索到的价格...")
- 返回: 价格区间 + 报价单引导

7.2 memory（长期记忆管理）

触发条件：

- 用户表达偏好、习惯、风格倾向
- 重要决策或技术选型
- 用户分享个人信息
- 解决复杂问题后有值得记录的经验
- 用户说"记住这个"

执行流程：

1. 读取 `agents/{角色}/MEMORY.md` 已有内容
2. 分析对话，提取偏好/决策/上下文/经验/个人信息
3. 按模板追加到 `## 记忆记录` 区域

记录模板：

YYYY-MM-DD HH:MM - 简短标题

- ****偏好****：用户的偏好和习惯
- ****决策****：重要的决策及理由
- ****上下文****：项目或对话的背景信息
- ****经验****：学到的经验和教训
- ****个人信息****：用户的称呼、角色、背景

7.3 xufeng-material-navigator（产品官网导航）

触发条件：

- 用户询问某牌号详情
- 用户询问产品品类
- 用户询问公司简介、联系方式
- 用户询问牌号对照表、规格表

核心功能：

- 自动匹配 32 款产品到对应的官网页面 URL
- 调用 `agent_browser` 工具触发前端页面导航
- 大小写不敏感的牌号匹配

8. 工具 (Tools) 体系

8.1 agent_browser（官网导航）

```
async def agent_browser(url: str, content: str) -> ToolResponse:
```

用途： 导航用户浏览器到旭丰新材料官网的指定页面。

参数：

- `url` — 相对路径（如 `/products/hot-work/h13/`）或完整 URL
- `content` — 页面简介（产品硬度、典型应用等）

实现原理： 返回 `ToolResponse` 并附带 `metadata: {action: "page_navigation", payload: {url, content}}`，由 `MetatoolReActAgent._acting()` 将 `metadata` 复制到 `Msg` 上，SSE 流式响应将其作为 `action` 事件发送给前端，前端据此导航。

8.2 web_search（MCP 网络搜索）

```
# 通过 HttpClient 注册
mcp_client = HttpClient(
    name="web_search",
    transport="streamable_http",
    url="http://47.94.179.114:3000/mcp",
)
await toolkit.register_mcp_client(mcp_client)
```

用途： 查询外部网络信息（市场价格、技术资料、行业标准）。

配置：

- MCP 服务器地址: `http://47.94.179.114:3000/mcp`
- 传输协议: Streamable HTTP
- 客户端类型: 无状态（每次调用创建临时会话）

注册位置：

- 主 Agent 的 toolkit（`agent.py` 第 60-66 行）
- 每个子智能体工具的 mini toolkit（`sub_agent_tools.py` 第 71-72 行）

8.3 文件工具

AgentScope 内置的三个文本文件操作工具：

```
from agentscope.tool import view_text_file, write_text_file,
insert_text_file
```

用途： memory 技能使用这些工具读写 `MEMORY.md` 文件。

工具	用途
<code>view_text_file</code>	读取文本文件内容
<code>write_text_file</code>	写入/覆盖文本文件
<code>insert_text_file</code>	追加内容到文本文件

8.4 子智能体工具 (sub_agent_tools.py)

三个专业子智能体工具，每个工具内部创建一个临时的 **ReActAgent**：

```
async def sales_agent_tool(question: str, context: str = "") ->
ToolResponse:
async def technical_agent_tool(question: str, context: str = "") ->
ToolResponse:
async def production_agent_tool(question: str, context: str = "") ->
ToolResponse:
```

内部实现（以 **sales_agent_tool** 为例）：

```
async def sales_agent_tool(question: str, context: str = "") ->
ToolResponse:
    # 1. 创建 mini Toolkit (web_search MCP + memory skill)
    toolkit = await _build_sub_agent_toolkit()

    # 2. 加载 sales 角色的系统提示词
    system_prompt = load_agent_prompt("sales")
    system_prompt += "\n\n## 长期记忆文件\n..." # 注入 memory 路径

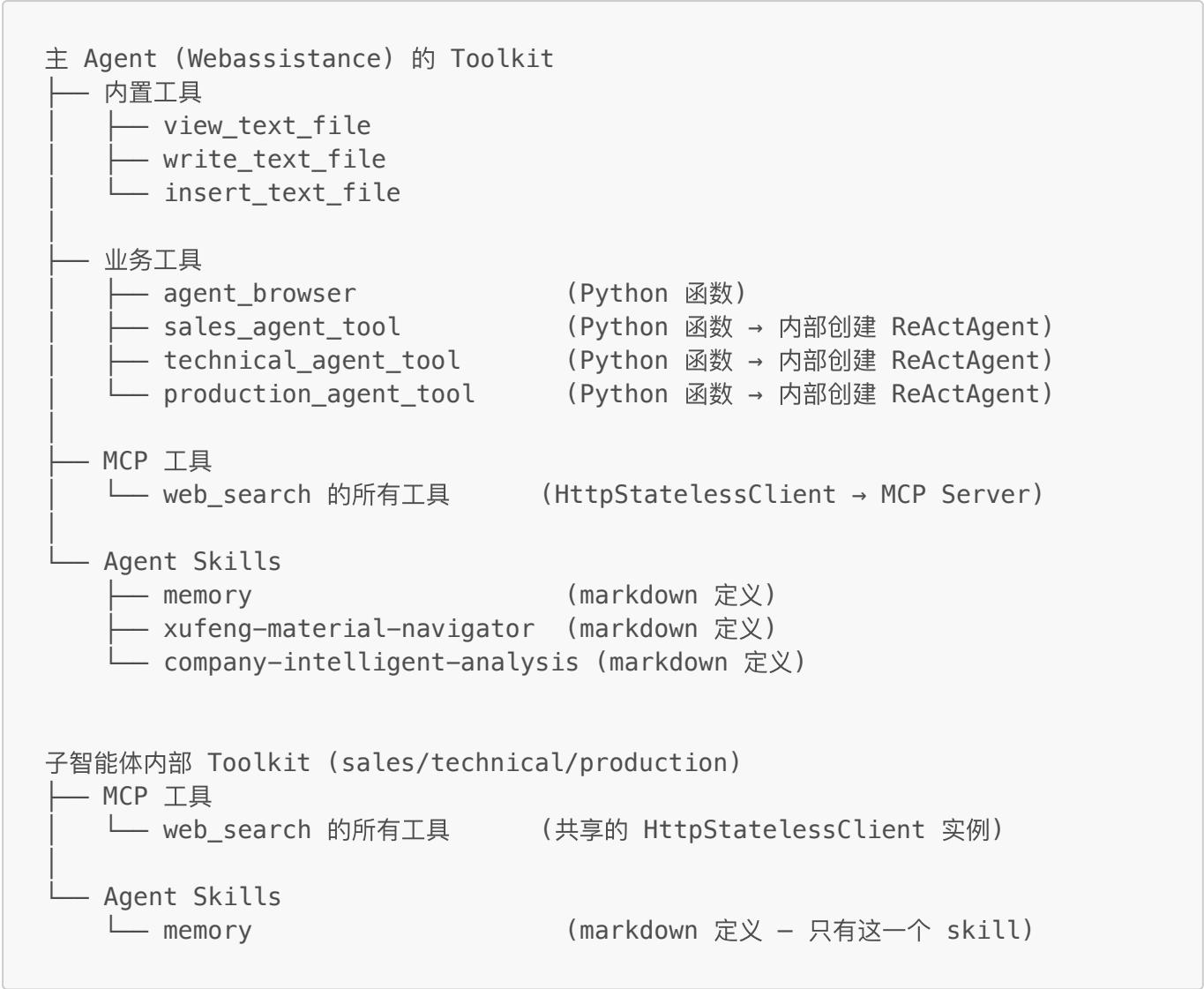
    # 3. 创建临时 ReActAgent
    agent = ReActAgent(
        name="SubAgent_sales",
        model=_get_model(), # OpenAIChatModel (stream=False)
        sys_prompt=system_prompt,
        memory=InMemoryMemory(), # 临时记忆，用完即丢
        toolkit=toolkit,
        formatter=OpenAIChatFormatter(),
    )

    # 4. 发送消息，等待回复，丢弃 Agent
    msg = Msg("user", question, "user")
    reply = await agent(msg)
    return ToolResponse(
        content=[TextBlock(type="text", text=reply.get_text_content())],
        is_last=True,
    )
```

设计要点：

- **Stateless** — 每次调用创建新的 **ReActAgent**，用完即 GC
- **独立 Toolkit** — 每个子智能体有自己的 mini Toolkit，只含 web_search MCP + memory skill
- **共享 Model** — **OpenAIChatModel** 实例在模块级别缓存，避免重复创建
- **共享 MCP Client** — **HttpStatelessClient** 实例在模块级别缓存
- **无 stream** — 内部调用不需要流式输出，**stream=False**

8.5 工具注册全景图



9. API 接口文档

详见 [BE/API.md](#), 以下为摘要:

9.1 健康检查

GET /health
→ { "status": "healthy", "active_sessions": 3 }

9.2 认证

POST /auth
Body: { "api_key": "sk-frontend-001" }
→ { "session_id": "uuid", "user_id": "user_001", "expires_at": "ISO8601" }

有效 API Keys: **sk-frontend-001** (user_001), **sk-frontend-002** (user_002)

9.3 同步对话

```
POST /chat
Body: { "session_id": "uuid", "message": "你好" }
→ { "session_id": "uuid", "content": "回复", "role": "assistant" }
```

9.4 流式对话 (推荐)

```
POST /chat/stream
Body: { "session_id": "uuid", "message": "你好" }

SSE 事件:
data: "文本累积更新"
data: { "type": "action", "action": "page_navigation", "payload": {...} }
data: [DONE]
```

9.5 登出

```
POST /logout
Body: { "session_id": "uuid" }
→ { "status": "logged out" }
```

10. 测试

10.1 测试脚本

[BE/test/test_agent.py](#) 提供完整的 API 集成测试：

```
# 基础测试 (连接 http://localhost:8088)
python BE/test/test_agent.py

# 自定义参数
python BE/test/test_agent.py --url http://localhost:8080 --key sk-frontend-001

# 快速测试 (只测 health + auth + chat)
python BE/test/test_agent.py --quick
```

10.2 测试覆盖

测试	说明
<code>test_health_check</code>	健康检查

测试	说明
test_auth	有效 Key 认证
test_invalid_auth	无效 Key (预期 401)
test_chat	同步对话
test_multi_turn_chat	多轮对话 (3 轮)
test_stream_chat	流式 SSE 对话
test_without_session	无 session_id 请求
test_logout	登出
test_session_reuse	同一 session 复用

11. 部署

11.1 Docker 部署（推荐）

```
cd BE

# 设置环境变量
cp .env.example .env
# 编辑 .env, 填入 DEEPSEEK_API_KEY 等

# 构建镜像
docker build -t agentscope-agent .

# 启动
docker-compose up -d
```

docker-compose.yml 环境变量：

变量	默认值	说明
DEEPSEEK_API_KEY	(必填)	DeepSeek API Key
DEEPSEEK_BASE_URL	https://api.deepseek.com	API 地址
MODEL_NAME	deepseek-chat	模型名
AGENT_ROLE	default	Agent 角色
PORT	8080	服务端口

11.2 手动部署

```
cd BE
```

```
# 安装依赖
pip install -r requirements.txt

# 配置环境变量
export DEEPSEEK_API_KEY=sk-xxx
export AGENT_ROLE=Webassistance

# 启动
python src/app.py
```

11.3 前端部署

```
cd FE/my-chat-app

# 安装依赖
npm install

# 开发模式
npm run dev          # → http://localhost:5173

# 生产构建
npm run build        # → dist/ 目录

# 预览构建结果
npm run preview
```

12. 开发指南

12.1 添加新的 Agent 角色

1. 在 `agents/` 下创建新目录（如 `agents/my-role/`）
2. 创建 5 个文件：
 - `PROFILE.md` — 身份定义
 - `SOUL.md` — 行为原则
 - `AGENTS.md` — 工作规则
 - `MEMORY.md` — 复制模板
 - `BOOTSTRAP.md` — 复制模板
3. 在 `.env` 中设置 `AGENT_ROLE=my-role`
4. 重启服务

12.2 添加新的 Skill

1. 在 `skills/` 下创建新目录（如 `skills/my-skill/`）
2. 创建 `SKILL.md`，包含 frontmatter：

```

---
name: my-skill
description: 技能描述
---
# 技能内容

```

3. 重启服务（skill 目录会自动扫描注册）

12.3 添加新的 Tool

1. 在 `tools/` 下创建 Python 文件
2. 实现异步函数，返回 `ToolResponse`:

```

async def my_tool(param: str) -> ToolResponse:
    """工具描述"""
    return ToolResponse(
        content=[TextBlock(type="text", text=f"结果: {param}")],
        is_last=True,
    )

```

3. 在 `agent.py` 的 `_create_toolkit()` 中注册:

```

from tools.my_module import my_tool
toolkit.register_tool_function(my_tool)

```

12.4 添加 MCP 工具

```

from agentscope.mcp import HttpStatelessClient

client = HttpStatelessClient(
    name="my_mcp",
    transport="streamable_http",
    url="http://example.com/mcp",
)
await toolkit.register_mcp_client(client)

```

12.5 项目启动清单

```

# 1. 后端
cd BE
cp .env.example .env # 编辑填入 API Key
pip install -r requirements.txt
python src/app.py # → http://localhost:8080

```

```
# 2. 前端 (另一个终端)
cd FE/my-chat-app
npm install
npm run dev          # → http://localhost:5173

# 3. 验证
curl http://localhost:8080/health
# → {"status":"healthy","active_sessions":0}

# 4. 测试对话
curl -X POST http://localhost:8080/auth \
  -H "Content-Type: application/json" \
  -d '{"api_key":"sk-frontend-001"}'
# → {"session_id":"xxx",...}

curl -X POST http://localhost:8080/chat \
  -H "Content-Type: application/json" \
  -d '{"session_id":"xxx","message":"你好"}'
# → {"session_id":"xxx","content":"你好! ...","role":"assistant"}
```

版本: 2026-05-24

技术栈: AgentScope + FastAPI + DeepSeek + React + TypeScript + Docker

维护团队: 黄石旭丰新材料科技有限公司